SmartMatch — Matching Process and Test Results

Application revision: `4fc80d0` (the revision on the class VM) · **Test dates:** 27–29 October 2026 · **Class URL:** https://exercise.smartmatch.cloud/exercise

1. Purpose

SmartMatch is a two-round classroom exercise for AI in Marketing. Each team chooses which 60 of 300 fictional students get a personal CBACH email for an event. Round 1 is Northline Analytics (technology). Round 2 is Harbor Consumer Brands (International Business and Marketing). All 300 students also get the usual mass email, which is the comparison. The tests checked that a team can complete both rounds and that matching results are correct and repeatable.

2. Matching Method

Inputs. The instructor loads one data file: 300 fictional student profiles and 12 events (`SmartMatch_Student_Body_300_10022026.xlsx`). A profile has a major, class year, past events attended, and, for 70 students, a profile card with stated interests and a career goal. 230 students have no card.

Ranking. The team saves a setting: four whole-number weights (0–10) for major, interest, past events and career goal. The default is 3/3/2/2. The system scores each student as the weighted sum of the four fits and sorts by score. Ties go to the student with more known information, then to the senior, then to a fixed order stored in the data file. The top 60 form the team's list. The ranking contains no randomness.

Outcome. The system then simulates each of the 300 students:

1. Notices an email: 20 in 100 for the mass email; 60 in 100 for the CBACH email (listed students only).
2. Signs up, if noticed: 5 in 100, plus 35 if a true interest matches the event topic, 15 for a fitting career goal, 5 or 10 for past events (1+ or 3+), and 5 for the targeted major.
3. Attends, if signed up: 65 in 100.

"True interest" is hidden data, not the card. Each chance uses a SHA-256 hash of the team's stored seed, the event, the student and the step. Therefore the same team and list always give the same result. **List added** is the number of listed students who signed up and would not have signed up from the mass email alone.

Round 2. Before Harbor, the team chooses how to ask students without a card for one: a promise (30% respond), a small reward (55%), or required (80%, but 15% of students stop responding). New cards change only the information the ranking uses. The Harbor list uses the same rules. Each team runs results once per event; the database rejects a second run.

3. Test Method

- **Public test:** on 29 October 2026, 05:53–05:57 UTC, Team 6 ran the full exercise on https://exercise.smartmatch.cloud/exercise with the live data file "Oct2-Ann-300". A browser session recorded the full text of both results pages, and a reload showed the same numbers. The VM runs `4fc80d0` according to the last deploy log. An earlier public run (Team 1, 27 October) also completed both rounds; its sign-up counts were not captured.
- **Local test:** a local copy of the same revision (PostgreSQL 16, exercise API, web front end), driven by Playwright in Chromium. Team 1 and Team 2 worked at the same time in separate browser sessions. Negative tests: wrong code, short code, empty setting name, double send, page reload, deep-link reload.
- **Procedure (both tests):** the instructor opened results for both events. Each team entered with its code, saved the setting "Balanced" (3/3/2/2), sent the Northline list and read the results. It then chose "A small reward", asked students, and sent and read the Harbor list.
- **Matching check:** a script ran the real matching code on the shipped fixture (SHA-256 `57cb47e7…9866`) twice with fixed seed 20261027 and compared the outputs. Unit and golden tests ran with pytest; front-end tests ran with Vitest.

4. Test Results

Results as shown on the results pages. Each list has 60 students, and the room has 60 seats. All runs used the setting "Balanced" and, before Harbor, asked with "A small reward".

Run	Round	Already coming	Came because of list	Seats open	List signed up	Would sign up anyway	List added
Public, Team 6	1 Northline	13	5	42	15	4	11
Public, Team 6	2 Harbor	17	7	36	20	8	12
Local, Team 1	1 Northline	12	7	41	9	1	8
Local, Team 1	2 Harbor	13	8	39	15	6	9
Local, Team 2	1 Northline	18	4	38	16	8	8
Local, Team 2	2 Harbor	11	6	43	17	5	12

Already coming: attendees the list did not bring (pre-registered students who came, plus everyone the mass email alone brought). *Came because of list:* listed students who came and whom the mass email would not have brought. *List added:* the same idea for sign-ups; only 65 in 100 sign-ups attend, so it is higher than *came because of list*.

The three seat counts add up to 60 in every row, and *list added* equals *list signed up* minus *would sign up anyway*. For Team 6, the mass email alone gave 10 sign-ups (Northline) and 15 (Harbor); with the list there were 21 and 27, a difference of 11 and 12. Each team has its own seed, so the same setting gives different numbers for each team. Asking with a small reward reached the 230 students without a card; 127 completed a card and 0 stopped responding (55% of 230, rounded half up, is 127). For Team 6, Harbor's *list added* was 8 before asking and 12 after; "Major first" after asking gave 6. The fixed-seed script gave identical output twice; all four of its lists had 60 unique students in weight order.

5. What Worked

- The public URL served the full exercise. Team 6 entered with its code, completed Round 1, asked for more, completed Round 2, and saw both results pages. In the local test, two teams did this at the same time.
- Matching follows the documented rules. The code agrees with every rule in Section 2.
- Results persist: reloading the page keeps the run, and a double press of "Send list" recorded one run.
- Invalid input is rejected: a wrong team code shows "That code is not right for this team"; a short code and an empty setting name disable the button.
- Automated tests: 1,271 front-end tests, 66 golden exercise tests and 1,440 exercise unit tests passed.
- The last deploy (26 October, 06:32 UTC) placed `4fc80d0` on the VM, and all of its checks passed.

6. What Did Not Work

- **Deploy check (fixed).** On 26 October two deploys failed at the public check. The first used the retired `pilot.plated.blog` address. The second pointed the check at the exercise host, which does not serve the route the check expects. The third deploy, against `pilot.smartmatch.cloud`, passed. No automated check covers the exercise host yet.
- **Walkthrough recorder (open).** `tools/walkthrough/record_student_flow.mjs` does not enter the 4-digit team code, so it fails at step 1. This does not affect students.
- **Test environment limits (open).** The local API and front end ran as host processes on Python 3.12 (Docker Hub rate limit). `make setup` fails on Python 3.13. Two unit tests failed only because of a local settings file.
- **Not reproduced.** The "5,000 simulated runs" table in the earlier write-up has no script in the repository, so this report does not use it.
- **Not tested.** The public test was one team at a time; it did not test concurrent teams or invalid input on the public site. Main now has a newer revision (`763e666`, list size fixed at 60 in the database); it is not deployed and was not tested.

7. Conclusion

The full two-round exercise passed on the public URL (VM revision `4fc80d0`, Team 6 run) and in the local two-team test. Matching agrees with the documented rules, and results are repeatable with a fixed seed. No blocking defect was found. The main limits are a single-team public test and no automated health check for the exercise host.